

2004:64

*Chalmers tekniska högskola
Inst. för signaler och system. EX*

7
CTH

CHALMERS



Digital drum synthesis TRX for the Elektron Machinedrum SPS-1

DAVID MÖLLERSTEDT

*Signal Processing Group
Department of Signals and Systems
Chalmers University of Technology
Göteborg, Sweden, 2004*

EX064/2004

2.0 Innehållsförteckning

1.0	Abstract	
2.0	Innehållsförteckning	1
3.0	Introduktion	2
3.1	Bakgrund och inledning	2
3.2	Arbetsuppgiften	2
3.3	Problem och syfte	3
4.0	Plattform	4
4.1	Machinedrum SPS-1	4
5.0	Utvecklingsprocess	6
5.1	Analys av TR-808	6
5.2	Översikt grupper	7
5.2.1	En oscillator	7
5.2.2	Två oscillatorer	7
5.2.3	Brus gruppen	8
5.3	Modell	8
5.3.1	En oscillator	8
5.3.2	Två oscillatorer	9
5.3.3	Brus gruppen	10
6.0	Implementation	12
6.0.1	Sinus	12
6.0.2	Brus	12
6.0.3	Brus genererat från sex fyrkants vågor.	13
6.1	Anpassning av syntesmodeller	14
6.2	Svårigheter vid implementering	14
7.0	Resultat och slutsats	16
8.0	Litteraturförteckning	17
9.0	Appendix A	18
9.1	Gämförande studie TR-808 och SPS-1	18
10.0	Appendix B	20
10.1	DSP kod för TRX-CH	20

3.0 Introduktion

3.1 Bakgrund och inledning

Under nittonhundratalet introducerades elektroniska musikinstrument. På 60-talet tog utvecklingen fart i samband med framsteg inom elektroniken som möjliggjorde mindre och billigare konstruktioner. Det som vi i dag refererar till som synthar började serietillverkas.

De första syntharna var helt inriktade mot att låta så realistiskt som möjligt för att kunna ersätta de instrument som de härmade. På 70-talet skapades de första syntharna som var renodlade mot att göra trumljud, de kallades trummaskiner. De innehöll som regel också en sequenser för att kunna återspela en takt. Under 70-talet och 80-talet fortsatte utvecklingen av trummaskiner mot allt mer avancerade apparater.

Under mitten på 80-talet slog samplingstekniken igenom, den innebär att förinspelade ljud spelas upp med valfri hastighet. Samplingstekniken passar sig mycket bra i de fall då man vill härmna trumljud, då de är relativt korta och därför inte kräver orimligt mycket lagringsminne. I och med detta stannade utvecklingen av trummaskiner baserade på elektronisk synt upp.

Trots de många fördelar som finns med den samplingsbaserade tekniken finns det någonting musikaliskt tilltalande i de syntetiskt genererade trumljuden. Detta blev tydligt under 90-talet då efterfrågan ökade på de klassiska trummaskinerna som gjorts innan samplingstekniken slog igenom. De små skillnader som finns mellan varje genererat ljud jämfört med exakta repeterade uppspelningar av en sampling låter i ett musikaliskt sammanhang levande. En önskan att skapa trumljud som inte i första hand är realistiska blev också mer framträdande i samband med de nya musikstilar som växte fram. Då var de syntetiserade trumljuden som är baserade på elektriska kretsar mer lämpade än en sampling av en riktig trumma.

3.2 Arbetsuppgiften

Elektron ESI AB designade 2001 Machinedrum SPS-1 med avsikt att vara den mest avancerade trummaskinen som skapats. En av funktionerna var då att digitalt efterlikna de ljud som analoga syntetiska trummaskiner skapar. Som förebild valdes Roland TR-808. Roland skapade under 80-talet en handfull trummaskiner som ansågs mycket avancerade. Det var också trummaskiner från denna serie som först började återupptäckas när viljan att använda elektroniskt genererade trumljud som alternativ till samplingar tog fart. TR-808 är den sista helt analoga trummaskin som skapades i Rolands TR-serie. Totalt består TR-808 av 11 ljudgenererande kanaler, var och en skapad för att låta som en viss typ av trumma. För några kanaler är trumkaraktären valbar.

Avsikten med detta examensarbete är att skapa en digital synt som skall kallas TRX och skall ingå som en av de fyra syntestyper som ingår i Machinedrum SPS-1. TRX skall vara baserad på den ljudgenerering som finns i Roland TR-808. TRX besitta samma ljudkaraktär som TR-808 och skall vidare erbjuda utökade kontrollmöjligheter jämfört med den analoga förebilden. Implementeringen skall ske på Elektrons ESP plattform.

3.3 Problem och syfte

Syftet med TRX är att fånga de aspekter av den analoga syntetiska förlagan som uppfattas som musikaliskt tilltalande. I vissa fall återskapas det som en naturlig följd av att en modell av den analoga kretsen byggs. I andra fall måste modellen påverkas för att uppföra sig på önskat sätt. Uppgiften innehåller alltså en subjektiv bedömning av förlagan och den digitala modellen där i första hand de musikaliska aspekterna skall överföras.

Då en tillfredställande modell har skapats skall möjligheterna att utvidga den digitala modellen förbi de gränser som de analoga konstruktörerna hade utforskas. Det kan gälla utökade omfång för frekvens och tidskontroll över de genererade trumljuden.

I TR-808 bidrar vad som skulle kunna anses vara missljud till karaktären. Vid starka ljudvolymmer uppkommer distorsion som får trumljudet att låta mer kraftfullt. I den digitala modellen är det önskvärt att frikoppla denna effekt från volym inställningen. Det skall gå att ha distorsion på ett svagt ljud och det skall gå att generera ett kraftigt ljud utan distorsion.

En del i uppgiften är att skapa modeller som utgår ifrån de som uppskattas i TR-808 men i TRX addera egenskaper som ger ökade möjligheter att skapa trumljud som behåller samma trumkaraktär som förlagan. De ljud som skapas av TR-808 har på grund av sin historia på ett beprövat sätt funnit uppskattning. Vid utökning av de ljud som kan skapas krävs att den håller samma kvalitet.

Syntesmodellen skall implementeras i realtidsmiljö, vilket innebär att signalen inte kan uträknas i förväg utan måste leverera sina utdata inom förbestämd tid. Det finns alltså en absolut övre gräns för den tid och därmed också komplexitet som modellen kan ha. Uppgiften innebär att maximalt utnyttja den tillgängliga tiden för att skapa ett så liknande och musikaliskt användbart ljud som möjligt.

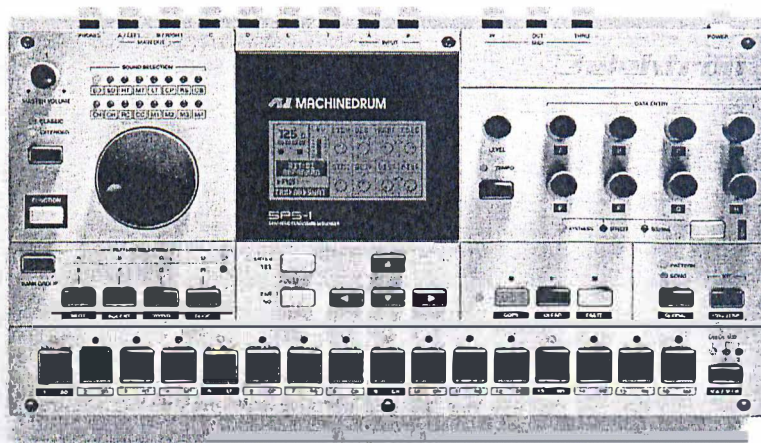
4.0 Plattform

4.1 Machinedrum SPS-1

Målplattformen för den syntesmodell som skall utvecklas är Elektrons Machinedrum SPS-1.

FIGUR 1.

Elektron Machinedrum SPS-1



Machinedrum är baserad på Elektrons ESP plattform. Den består av ett processorkort med en Motorola Coldfire 56206e 40MHz för MIDI, grafik för LCD skärm samt sekvenser baserade uppgifter. Vidare finns två Motorola DSP 56303 samt tillhörande minnessystem. Ett kretskort hanterar audio, där återfinns Crystal CS 4334 DA-omvandlare och Crystal CS 5331 AD-omvandlare kopplade till DSP-systemet via ett I2S gränssnitt. Detta kretskort hanterar de analoga in och utgångarna på Machinedrum SPS-1. Ett kretskort för att hantera användargränssnittet är kopplat via ett seriellt interface till Coldfireprocessorn. Där återfinns LCD och samtliga elektromekaniska komponenter.

TRX programmeras till största del i assembler för DSP-systemet, dock återfinns vissa delar som är nära knutna till sekvensern i C-kod på Coldfireprocessorn.

Ljudgenereringen för Machinedrums samtliga 16 parallella trumljud sker i en DSP varefter ljudet förs över digitalt till DSP nummer två där effekter läggs på samt mixning av ljuden sker.

Varje ljudgenererande enhet har en begränsad given tid för att leverera ett resultat. Vid varje anrop skall 32 sampels av audio data levereras, till detta finns maximalt 3200 klockcykler tillgängliga. Machinedrum SPS-1 är designad som ett hårt realtidssystem. Om exekveringstiden vid ett anrop överstiger 3200 klockcykler finns det risk att hela systemet hänger.

Plattform

Till förfogande för de ljudgenererande enheterna finns minne i form av 128 bytes minne för parametrar som behöver sparas mellan anrop, samt 512 bytes minne som delas med de andra syntes modellerna där innehållet inte sparas till nästa anrop.

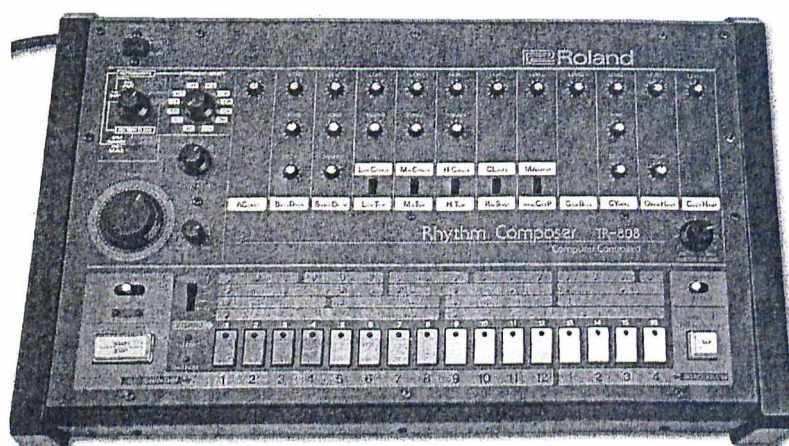
5.0 Utvecklingsprocess

5.1 Analys av TR-808

Till hjälp för skapandet av TRX tillhandahöll Elektron en styck TR-808 som under arbetet grundligt utforskades. Till en början genom att spela in, analysera och behandla de ljud som genererades. Denna del av analysen gick i första hand ut på att få en grundläggande förståelse för de ljud som genererades av TR-808. Längd, frekvensinnehåll och dynamisk förändring analyserades. Det framgick ganska snart att de 11 ljud som TR-808 genererade kunde delas upp i ett antal grupper. Ljud baserade på en grundton, ljud baserade på två grundtoner samt ljud baserade på filterat brus. Denna rapport kommer att behandla en specifik modell ur varje grupp. Det praktiska arbetet resulterade i 12 modeller, dock är många av modellerna så snarlika att det inte är intressant att separat redogöra för samtliga implementeringar.

FIGUR 2.

Roland TR-808



Som bas fanns även logiska kretsschema på TR-808. Genom analys av dessa kunde lämpliga mätpunkter väljas ut. I ett nästa steg användes dessa mätpunkter för att vidare analysera hur ljudsyntesen i TR-808 sker. Denna analys gav möjlighet att mäta på enskilda delar i de ljudgenererande kretsarna. Det gick att analysera ingående ljud komponenter innan amplitud enveloppen pålades. Detta möjliggjorde att frekvensbestämma delkomponenter i signaler. Samt att i de fall då det resulterande ljudet var för kort för en frekvensmätning genomföra mätningen på en alternativ punkt.

TABLE 1.

Ljud som kan genereras av TR-808 per spår

nr	typ	engelskt namn	svenskt namn
1	BD	bassdrum	bastrumma
2	SD	snaredrum	virveltrumma

TABLE 1.

Ljud som kan genereras av TR-808 per spår

nr	typ	engelskt namn	svenskt namn
3	LT/LC	low tom/low conga	lågt stämd puka/lågt stämd conga
4	MT/MC	mid tom/mid conga	mellan stämd puka/mellan stämd conga
5	HT/HT	high tom/high conga	högt stämd puka/högt stämd conga
6	RS/CL	rimshot/claves	kantslag/träpinnar
7	CP/MA	handclap/maracca	handklapp/maraccas
8	CB	cowbell	koklocka
9	CY	cymbal	cymbal
10	OH	open hihat	öppen hihat
11	CH	closed hihat	stängd hihat

Tre av kretsarna, de för låg, mellan och hög puka lämpade sig mycket väl för att kombineras till endast en modell. I TR-808 var inställningen för vilka frekvenser som kan väljas den skillnad som finns mellan kretsarna. I TRX kunde en modell med utökat frekvensområde skapas som kunde representera samtliga tre modeller. Samma förutsättning fanns för att kombinera de tre conga modellerna till en modell.

5.2 Översikt grupper

5.2.1 En oscillator

Typexemplet för denna grupp blir bastrumman. Det är också den medlem i gruppen som har flest inställningar.

TABLE 2.

Typ av trumma för TR-808 och motsvarande typ för TRX

TR-808 typ	TRX typ
BD	BD
LT/LC	XT XC
HT/HC	XT XC
HT/HC	XT XC

Ljudet börjar med ett karakteristiskt attack fas, speciell för varje modell i denna grupp, följt av en ton med bestämd tonhöjd. Där frekvensen som regel har en något sjunkande frekvens. I TR-808 är denna fast och inte kontrollerbar, i den digitala modellen styrs den med två kontroller. En för mängd frekvensfall och en för tiden för frekvensändring.

Den ton som genereras är sinusliknande. I den digitala modellen genereras den med en tabellerad sinus som moduleras till önskat frekvens innehåll.

5.2.2 Två ocsillatorer

Typexempel för denna grupp blir virveltrumma.

Utvecklingsprocess

TABLE 3.

Typ av trumma för TR-808 och motsvarande typ för TRX

TR-808 typ	TRX typ
SD	SD
RS	RS
CB/CL	CB CL

Ljudet byggs upp av två toner som via det inbördes intervallet skapar karaktären för trumman. I fallet virveltrumma inkluderas även en filtrerad brus signal som representerar sejarmattan i en fysisk trumma

5.2.3 Brus gruppen

Exemplifieras av stängd hi-hat. Ljudet byggs upp av brus från en brusgenerator, som är uppbyggd av sex fyrkants liknande funktions generatorer implementerade som elektriska vippor i TR-808. Bruset färgas sedan via filtrering.

TABLE 4.

Typ av trumma för TR-808 och motsvarande typ för TRX

TR-808 typ	TRX typ
CH	CH
OH	OH
CY	CY
MA	MA

5.3 Modell

5.3.1 En ocsillator

Vid undersökningarna av hur ljudet upplevs framkom att en stor del av karaktären på ljudet låg i hur attacken upplevdes. Valet föll därav på att sampla attackfasen och blanda in denna korta vågform med den syntetiserade ton som genererades från en sinus tabell. Vidare gavs möjlighet till att forma frekvensinnehållet, tonhöjd samt utklingning av tonen i trumman.

TABLE 5.

Kontroller för TRX-BD

nr		engelskt namn	svenskt namn
1	PTCH	pitch	frekvens
2	DEC	decay	utklingningstid
3	RAMP	frequency ramp	frekvens förändrings mängd
4	RDEC	frequency ramp decay	frekvens förändrings utklingningstid
5	STRT	start	hårdhet vid start

Utvecklingsprocess

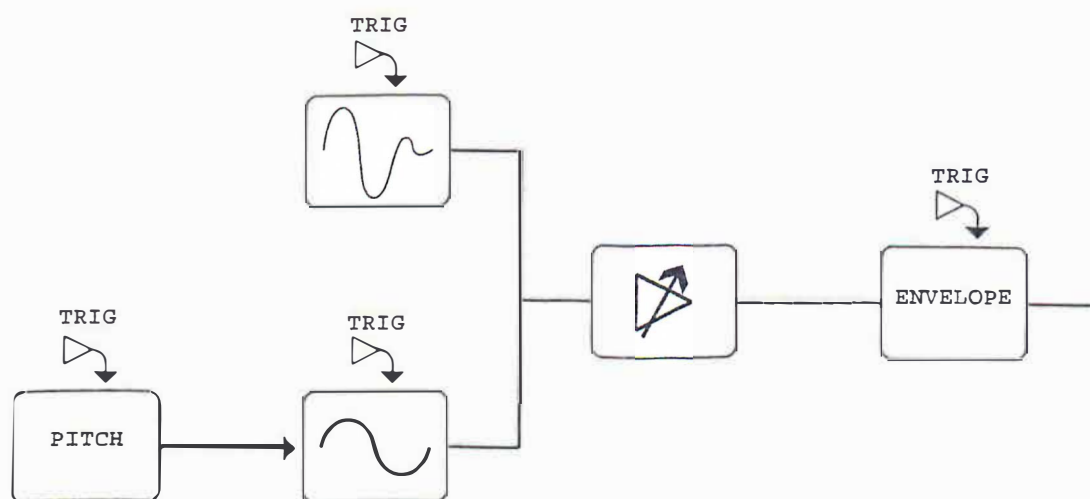
TABLE 5.

Kontroller för TRX-BD

nr		engelskt namn	svenskt namn
6	NOIS	noise	brus vid starten på ljudet
7	HARM	harmonic content	harmoniskt innehåll i ljudet
8	CLIP	clip	klippning av signalen

FIGUR 3.

Block diagram TRX-BD



5.3.2 Två ocsillatorer

Karaktären bestäms till mycket stor grad av intervallet mellan de två toner som ingår. Det visade sig att intervallet skulle vara konstant, även då en dynamisk frekvens ändring var önskad. Däremot var frekvens intervallet en mycket intressant parameter att ge användaren kontroll över.

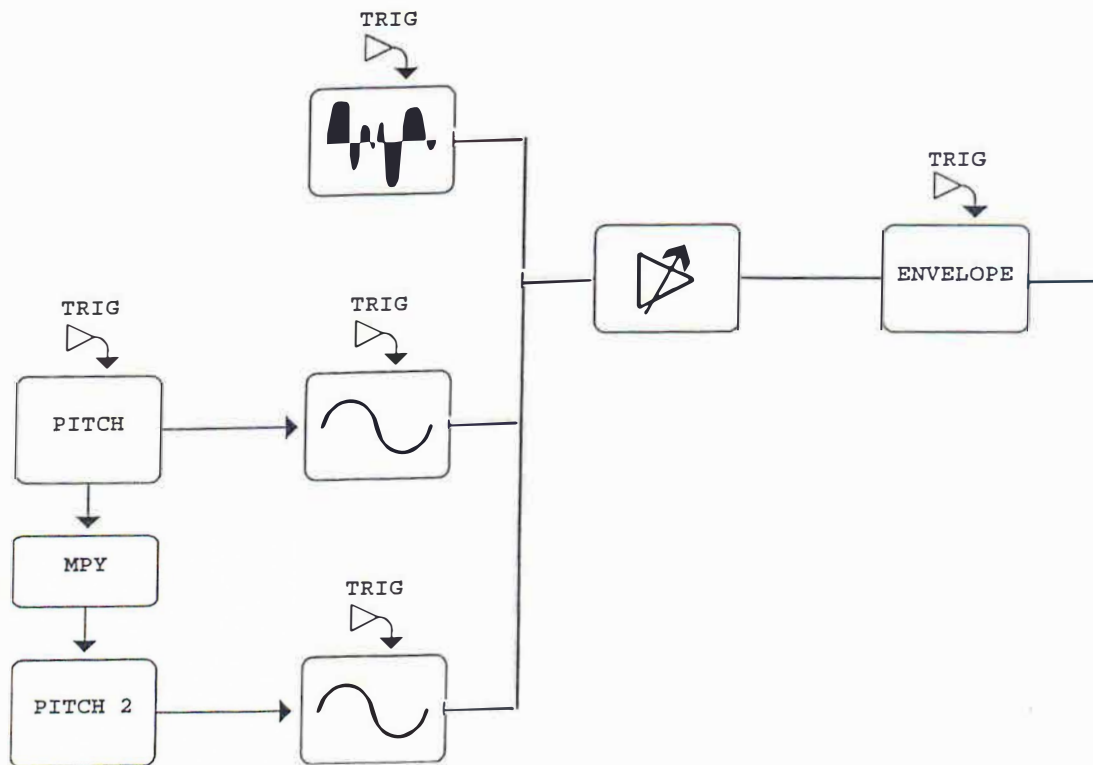
TABLE 6.

Kontroller för TRX-SD

nr		engelskt namn	svenskt namn
1	PTCH	pitch	frekvens
2	DEC	decay	utklingningstid
3	SNAP	snappy	special brus
4	NOIS	noise	brus
5	TONE	tone	volym balans mellan occilatorerna
6	TUNE	tuning	frekvens intervall mellan occilatorerna
7	BUMP	bump	frekvens ändring vid start
8	CLIP	clip	klippning av signalen

FIGUR 4.

Block diagram TRX-SD



5.3.3 Brus gruppen

Filtreringen av bruset har i denna grupp prioritet. De filter som krävs har inga krav på sig att vara faslinjära då signalen är brus. IIR filter valdes då de är implementerings mässigt fördelaktiga. Lågpasfiltering sker med ett 24 dB/oktav brant

TABLE 7.

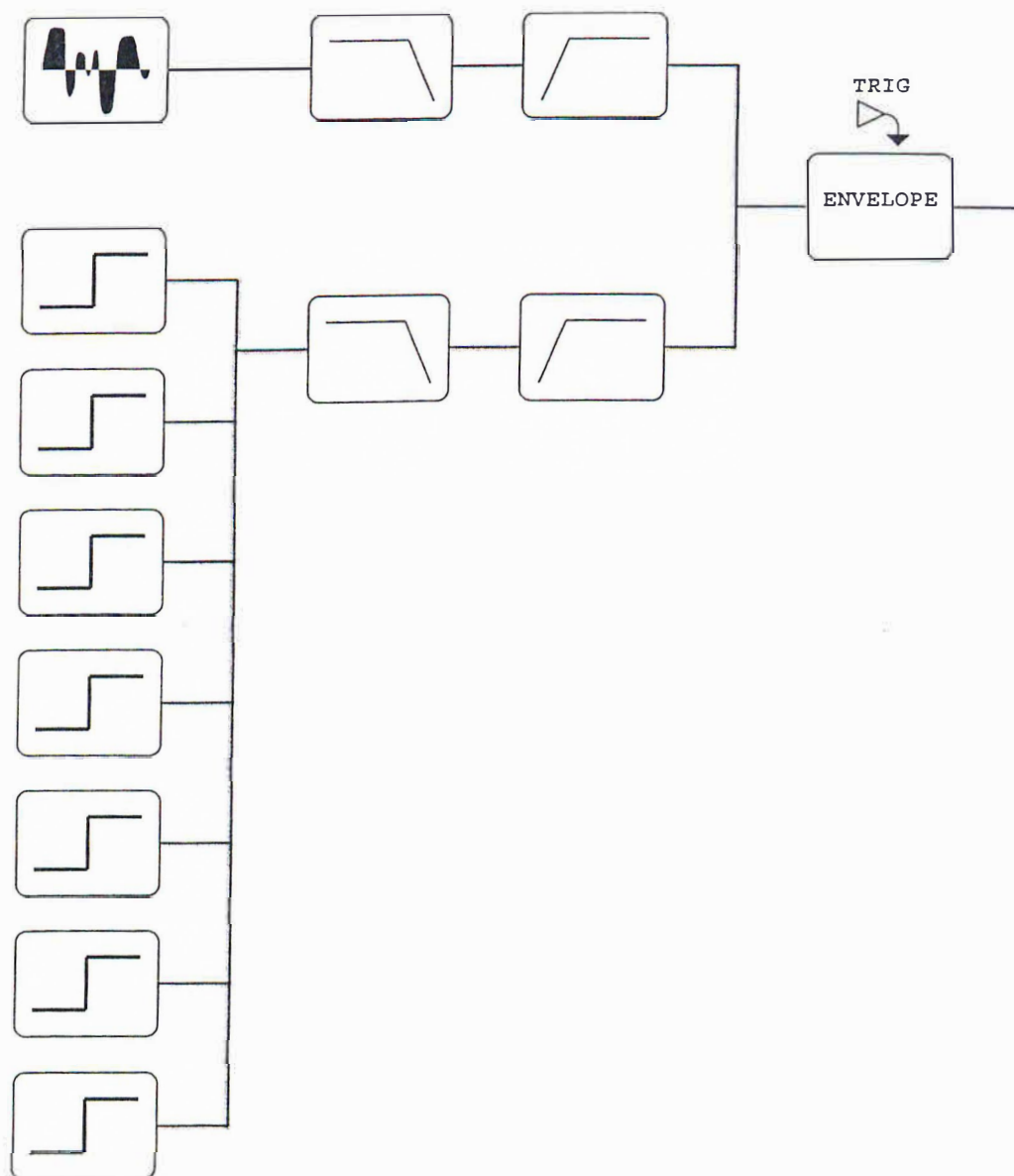
Kontroller för TRX-CH

nr		engelskt namn	svenskt namn
1	GAP	gap	mellanrum mellan hihat locken
2	DEC	decay	utklingningstid
3	LFP	lowpass frequency	lågpass frekvens
4	HPF	hipass frequency	högpass frekvens
5	PEAK	peak	resonans för hög och lågpas filter
6	MTAL	metal	metallisk klang i ljudet

filter där såväl brytfrekvens som resonans är kontrollerbart av användaren. Högpassfiltering sker med två 12 dB/oktav branta filter, de ena filtret är fast för att i alla lägen ta bort en viss mängd av basfrekvenser, de andra filteret är svepbart.

FIGUR 5.

Block diagram TRX-CH



6.0 Implementation

6.0.1 Sinus

Samtliga toner som används är genererade från en 24-bitars sinustabell på 4096 punkter. Storleken på sinustabellen var en given designparameter då den används av ett flertal delar av Machinedrum SPS-1 samt ligger i delat minne som begränsar dess storlek.

TABLE 8.

Kod för sinustabell

```
move y:(r6+BDX808_OSC1POS),b ; get position
move y:(r6+BDX808_OSC1POS+1),b0
move a1,x1
move a0,x0

do #32,bdx808_sin_loop
and #32768-1,b ; mask out relevant bits
move b1,r2 ; do this now to prevent pipeline stall
add x,b ; calculate new osc1pos
nop
nop
move x:(r2+sintab),a ; fetch sample
asr #2,a,a
nop
move a,x:(r4)+ ; do the actual saving

bdx808_sin_loop

move b1,y:(r6+BDX808_OSC1POS) ; save position
move b0,y:(r6+BDX808_OSC1POS+1); save position
```

6.0.2 Brus

Brus genereras lokalt via en slumpgenerator. I Machinedrum SPS-1 kan man köra flera parallella instanser av samma modell. Hade bruset genererats centralt hade korrelationen gjort att det inte hade upplevts som flera trummor utan som en volymökning. Samma argument gäller för de modeller som har flera bruskällor internt.

TABLE 9.

Kod för brus

;; calculate 32 samples of noise and place in free_x

```
move #>free_x+CH808_NOISE_BLOCK,r5
move #31,m5

move y:(r6+CH808_NOISE_A),b
move y:(r6+CH808_NOISE_B),x1

do #31,ch808_noise_loop
add x1,b ; "Rotate"
move x1,x0 ;x1 -> temp
```

Implementation

```

move      b1,x:(r5)+      ; do the actual saving
move      b1,x1           ; b1 -> x1
move      x0,b1           ; temp -> b1
ch808_noise_loop

add        x1,b            ; dont forget to do the last add
move x1,y:(r6+CH808_NOISE_A) ; save seeds while "rotating" them
move b1,y:(r6+CH808_NOISE_B)
move b1,x:(r5)+          ; do the actual saving

; we could take the noise in reverse order but this feels 'stable'
; we're done with the noise gen

```

6.0.3 Brus genererat från sex fyrkants vågor.

För att ge bruset rätt karaktär var det viktigt att skapa det på samma sätt som i originalet. Först prövades en modell där sinusvågor klipptes till fyrkantsvågor. Det gav ett resultat som lät mycket likt originalet. Tyvärr var det allt för processorintensivt för att kunna användas i den slutgiltiga implementeringen. Då det inte fanns plats i minnet för en fyrkants tabell av samma storlek som sinustabellen fick en alternativ metod skapas. Lyckligtvis var frekvenserna fasta och en tabell av minimal storlek kunde användas.

TABLE 10.

Kod för fyrkantsvågor repeterad 6 gånger

```

move y:(r6+CH808_COUNT2),b ; Load current pos
move #sqr_table2,y0        ; Load table pos
add y0,b                   ; calc effective adress
move y:(r6+CH808_OUT2),x0  ; Load current out-value
move b,r3                  ; load r3 with adress
move #SQR_FREQ2-1,m3

do #16,ch808_loop2
move      x:(r3)+,x1        ; load from table
move      x:(r3)+,y1        ; load from table
mpy       x0,x1,a           x:(r4)+,x1 ; calc out value & load old sum
mpy       x0,y1,b           x:(r4)+,y1 ; calc out value & load old sum
add       x1,a
add       y1,b
move      a,x:(r5)+         ; Save out
move      b,x:(r5)+         ; Save out
ch808_loop2
move r3,b                   ; load new adress
sub y0,b                    ; calc pos, table pos already in y1
nop
move b,y:(r6+CH808_COUNT2) ; Save current pos

```

TABLE 11.

Tabell frekvenser fyrkantsvågor

```

; sqr freqs
SQR_FREQ1 equ 114 ; this is for approx. freq 306 @ 44100
SQR_FREQ2 equ 102 ; values are for a whole period432

```

Implementation

SQR_FREQ3	equ 86	512
SQR_FREQ4	equ 80	551
SQR_FREQ5	equ 58	743
SQR_FREQ6	equ 52	826

6.1 Anpassning av syntesmodeller

Ur de tre huvudgrupperna skapades totalt 12 syntes modeller.

TABLE 12.

TRX

nr	typ	engelskt namn	svenskt namn
1	BD	bassdrum	bastrumma
2	SD	snaredrum	virveltrumma
3	XT	toms	pukor
4	XC	congas	kongas
5	RS	rimshot	kantslag
6	CP	handclap	handklapp
7	CL	maracca	träpinnar
8	CB	cowbell	koklocka
9	CH	closed hihat	stängd hihat
10	OH	open hihat	öppen hihat
11	CY	cymbal	cymbal
12	MA	maracca	maraccas

Samtliga modeller gavs sedan ett volymtillskott vid tidsmässigt närliggande triggas. Detta genom att eventuell volym som ej avklingat från tidigare tillslag adderades till en grundvolym som ligger på 85% av maximal volym.

Vid integrationen av syntesmodellerna till den miljö där de skall exekveras styrda av sequensern i Machinedrum SPS-1 krävdes ytterligare anpassning. De parametrar som går att kontrollera via sequensern måste beaktas så att de får önskad effekt i varje del i trigfaserna. Det innebär att samtliga användarstyrda parametrar måste kontinuerligt uppdateras, det är inte tillräckligt att läsa in parametrar vid trigögonblicket.

6.2 Svårigheter vid implementering

Generering av de sex fyrkantsvågorna visade sig vara för processorintensivt enligt den ursprungliga tekniken med hårdklippta sinus vågor. Lyckligtvis gav metoden med tabellerade fyrkantsvågor ett fullgott resultat. Denna metod kunde med fördel användas då frekvenserna låg fasta. Hade rörliga frekvenser varit ett krav hade en annan metod behövt användas.

Vid återstart av en trumma som är baserad på en sinusvåg och inte har klingat ut kan ett knäpp uppstå om vågformen startas om. Det är önskvärt i de flesta fall att återstarta vågformen, annars upplevs som att slaget inte riktigt träffar. Speciellt gäller

Implementation

det vid korta slag vid låg frekvens då bara en kort del av vågformen spelas upp innan ljudet har klingat ut. För att undvika missljud vid återtrig och kunna återstarta vågformen infördes en utklingningsfas innan slaget återstartades. En mycket snabb volym sänkning genomfördes, dock med det ursprungliga volym värdet sparat för att kunna bidra till volymen på nästa slag enligt ovan.

7.0 Resultat och slutsats

TRX är en digital modell baserad på den analoga elektriska förebilden TR-808. TRX är utökad i hänseende på kontrollmöjligheter. Ljudkaraktären från den klassiska förebilden i kombination med de nya möjligheterna till ljudskulptering är den aspekt av TRX som har fått mest uppskattning.

Trots de olika tekniker som har använts internt har TRX för användaren ett homogent framträdande.

För en jämförande studie se Appendix A. TR-808 har vid denna jämförelse ställts med de kontroller som finns i mittläge, SPS-1 har ställts in för att ha ett liknande ljud.

I kombination med de andra syntestyperna har Machinedrum SPS-1 blivit en mycket komplett och uppskattad produkt. Den har sedan försäljningsstarten 2001 fram till dagens datum sålt 1481 enheter.

8.0 Litteraturförteckning

TR-808 Schematics, Roland Corporation, 15 Juni 1981

Motorola, DSP56300 Family Manual, Order number DSP56300FAM/AD.

Watkinson John, The Art of Digital Audio, Focal Press, ISBN: 02405187.

Oppenheim A. V. & Schaffer R. W. 1998. Discrete-time Signal Processing. ISBN 0-13-754920-2

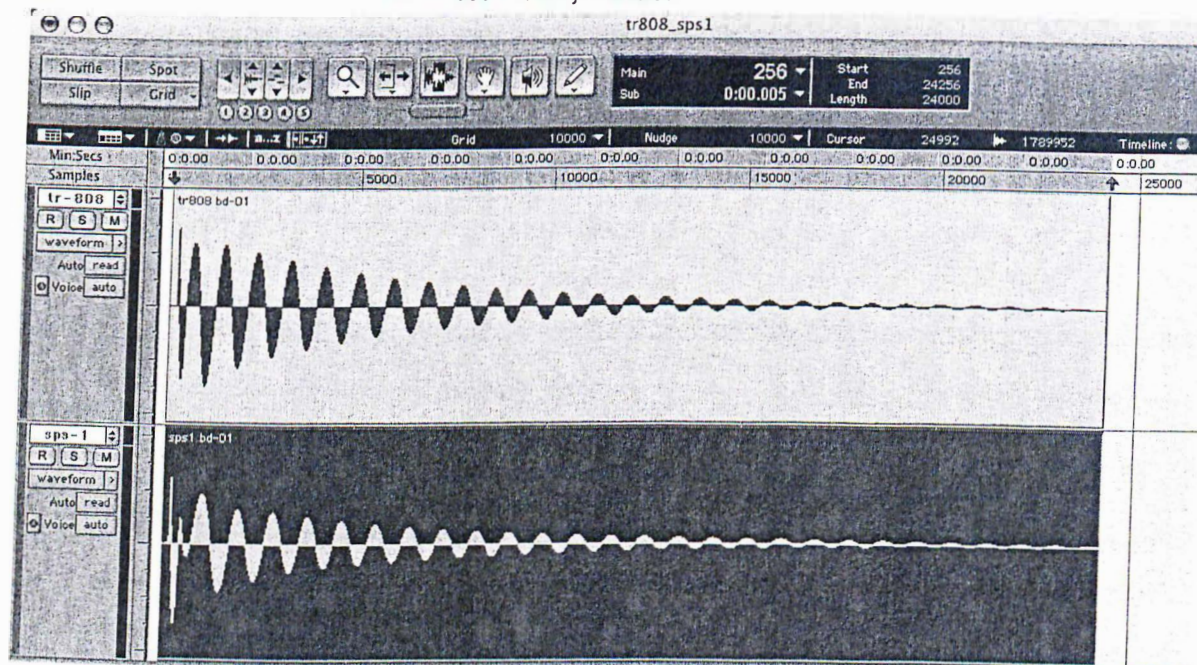
The Complete MIDI 1.0 Detailed Specification, Document version 96.1, The MIDI Manufacturers Association

9.0 Appendix A

9.1 Jämförande studie TR-808 och SPS-1

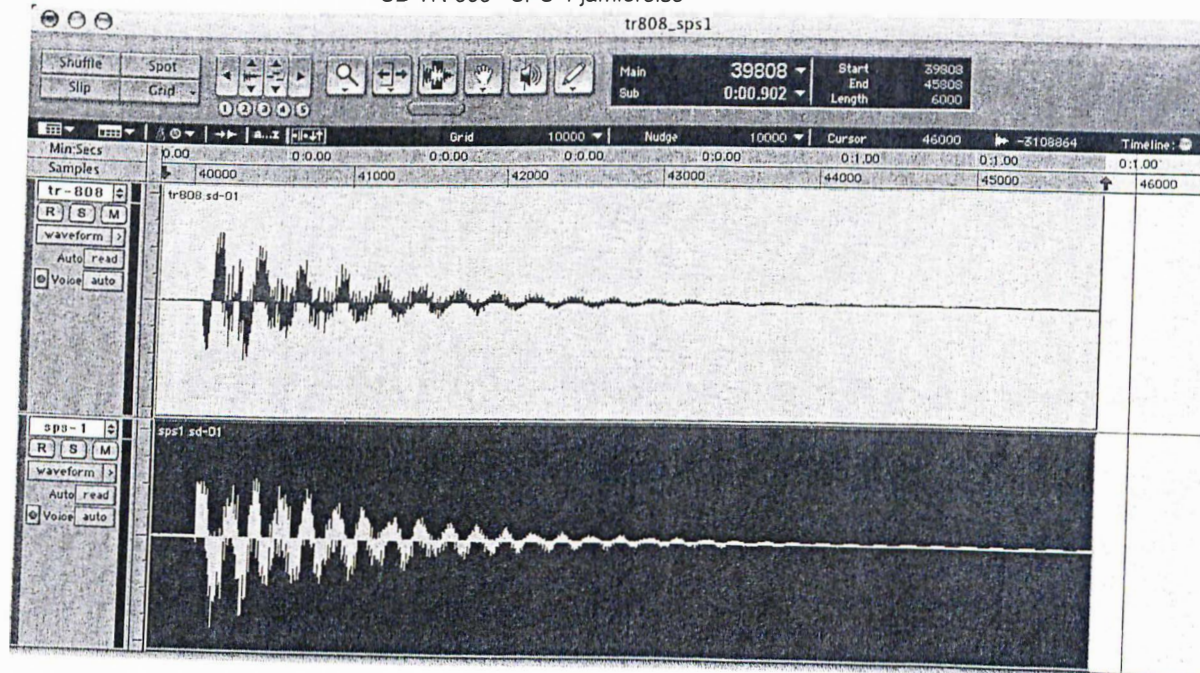
FIGUR 6.

BD TR-808 - SPS1 jämförelse



FIGUR 7.

SD TR-808 - SPS-1 jämförelse

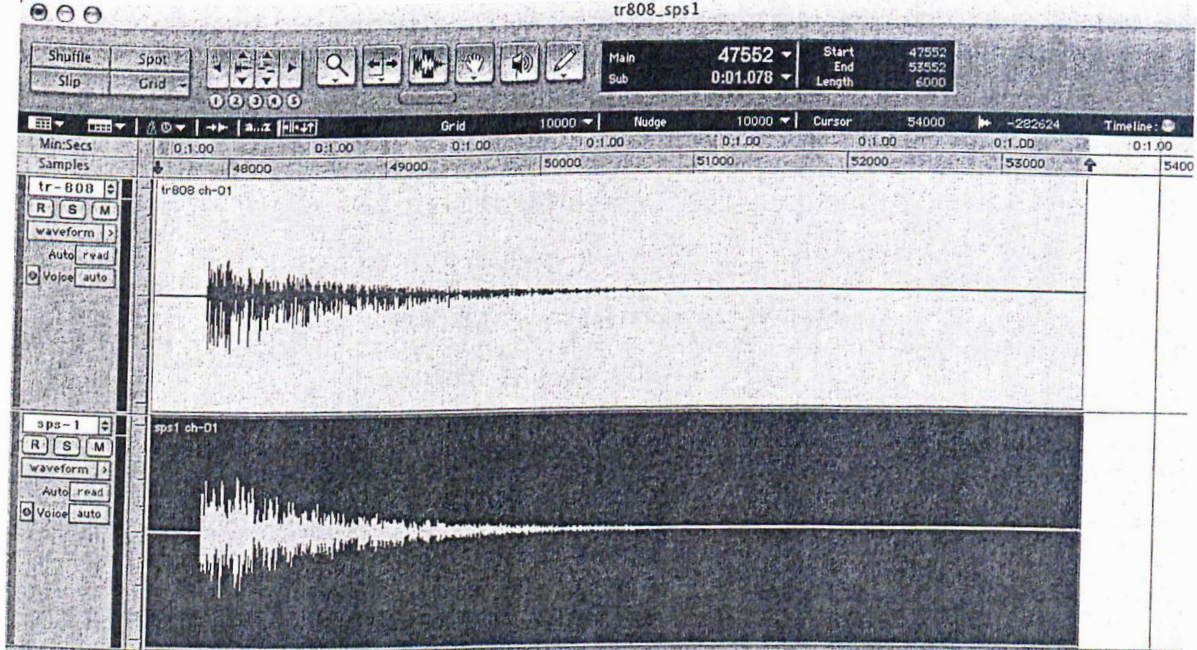


Appendix A

FIGUR 8.

CH TR-808 - SPS-1 jämförelse

tr808_sps1



10.0 Appendix B

10.1 DSP kod för TRX-CH

```

; Calculate a buffer of 32 sound data
;
; r6 = pointer to data
; r7 = pointer to sound data

tr808_ch_plugin

;; calculate 32 samples of noise and place in free_x

move #>free_x+CH808_NOISE_BLOCK,r5
move #31,m5

move y:(r6+CH808_NOISE_A),b
move y:(r6+CH808_NOISE_B),x1

do #31,ch808_noise_loop
add x1,b                                ; "Rotate"
move x1,x0                              ;x1 -> temp
move b1,x:(r5)+                          ; do the actual saving
move b1,x1                              ;b1 -> x1
move x0,b1                              ;temp -> b1
ch808_noise_loop

add x1,b                                ; dont forget to do the last add
move x1,y:(r6+CH808_NOISE_A)            ; save seeds while "rotating" them
move b1,y:(r6+CH808_NOISE_B)
move b1,x:(r5)+                          ; do the actual saving
; we could take the noise in revers order but this feels 'stable'
; we're done with the noise gen

;;; apply volume to the noise to control the dist in the lp ;;;

move y:(r6+CH808_NOISE_VOL),x0

do #16,ch808_vol
move x:(r5)+,y0
move x:(r5)-,y1
mpy y0,x0,a                              ; apply vol
mpy y1,x0,b                              ; apply vol
move a,x:(r5)+                          ; save sample
move b,x:(r5)+                          ; save sample
ch808_vol

;;; vol done ;;;

;..... do six sqrs ;.....

```

Appendix B

```
move #>free_x+CH808_NOISE_BLOCK,r4
; both r4 and r5 points to SQR_BLOCK
move #31,m4

,,,,,,,,, no 1 :,,,,,,,,,

move y:(r6+CH808_COUNT1),b           ; Load current pos
move #sqr_table1,y0                 ; Load table pos
add y0,b                             ; calc effective adress
move y:(r6+CH808_OUT1),x0           ; Load current out-value
move b,r3                           ; load r3 with adress
move #SQR_FREQ1-1,m3

do #16,ch808_loop1
move x:(r3)+,x1                     ; load from table
move x:(r3)+,y1                     ; load from table
mpy x0,x1,ax:(r4)+,x1               ; calc out value & load old sum
mpy x0,y1,bx:(r4)+,y1               ; calc out value & load old sum
add x1,a
add y1,b
move a,x:(r5)+                       ; Save out
move b,x:(r5)+                       ; Save out
ch808_loop1
move r3,b                           ; load new adress
sub y0,b                            ; calc pos, table pos already in y1
nop
move b,y:(r6+CH808_COUNT1)          ; Save current pos

,,,,,,,,, no 2 :,,,,,,,,,

move y:(r6+CH808_COUNT2),b           ; Load current pos
move #sqr_table2,y0                 ; Load table pos
add y0,b                             ; calc effective adress
move y:(r6+CH808_OUT2),x0           ; Load current out-value
move b,r3                           ; load r3 with adress
move #SQR_FREQ2-1,m3

do #16,ch808_loop2
move x:(r3)+,x1                     ; load from table
move x:(r3)+,y1                     ; load from table
mpy x0,x1,ax:(r4)+,x1               ; calc out value & load old sum
mpy x0,y1,bx:(r4)+,y1               ; calc out value & load old sum
add x1,a
add y1,b
move a,x:(r5)+                       ; Save out
move b,x:(r5)+                       ; Save out
ch808_loop2
move r3,b                           ; load new adress
sub y0,b                            ; calc pos, table pos already in y1
nop
move b,y:(r6+CH808_COUNT2)          ; Save current pos

,,,,,,,,, no 3 :,,,,,,,,,
```

Appendix B

```
move y:(r6+CH808_COUNT3),b      ; Load current pos
move #sqr_table3,y0             ; Load table pos
add y0,b                        ; calc effective adress
move y:(r6+CH808_OUT3),x0        ; Load current out-value
move b,r3                       ; load r3 with adress
move #SQR_FREQ3-1,m3

do #16,ch808_loop3
move x:(r3)+,x1                  ; load from table
move x:(r3)+,y1                  ; load from table
mpy x0,x1,ax:(r4)+,x1            ; calc out value & load old sum
mpy x0,y1,bx:(r4)+,y1           ; calc out value & load old sum
add x1,a
add y1,b
move a,x:(r5)+                   ; Save out
move b,x:(r5)+                   ; Save out
ch808_loop3
move r3,b                        ; load new adress
sub y0,b                         ; calc pos, table pos already in y1
nop
move b,y:(r6+CH808_COUNT3)      ; Save current pos

,,,,,,,,, no 4 ,,,,,,,,,,

move y:(r6+CH808_COUNT4),b      ; Load current pos
move #sqr_table4,y0             ; Load table pos
add y0,b                        ; calc effective adress
move y:(r6+CH808_OUT4),x0        ; Load current out-value
move b,r3                       ; load r3 with adress
move #SQR_FREQ4-1,m3

do #16,ch808_loop4
move x:(r3)+,x1                  ; load from table
move x:(r3)+,y1                  ; load from table
mpy x0,x1,ax:(r4)+,x1            ; calc out value & load old sum
mpy x0,y1,bx:(r4)+,y1           ; calc out value & load old sum
add x1,a
add y1,b
move a,x:(r5)+                   ; Save out
move b,x:(r5)+                   ; Save out
ch808_loop4
move r3,b                        ; load new adress
sub y0,b                         ; calc pos, table pos already in y1
nop
move b,y:(r6+CH808_COUNT4)      ; Save current pos

,,,,,,,,, no 5 ,,,,,,,,,,

move y:(r6+CH808_COUNT5),b      ; Load current pos
move #sqr_table5,y0             ; Load table pos
add y0,b                        ; calc effective adress
move y:(r6+CH808_OUT5),x0        ; Load current out-value
move b,r3                       ; load r3 with adress
```

Appendix B

```
move #SQR_FREQ5-1,m3

do #16,ch808_loop5
move x:(r3)+,x1          ; load from table
move x:(r3)+,y1          ; load from table
mpy x0,x1,ax:(r4)+,x1    ; calc out value & load old sum
mpy x0,y1,bx:(r4)+,y1    ; calc out value & load old sum
add x1,a
add y1,b
move a,x:(r5)+           ; Save out
move b,x:(r5)+           ; Save out
ch808_loop5
move r3,b                ; load new adress
sub y0,b                 ; calc pos, table pos already in y1
nop
move b,y:(r6+CH808_COUNT5) ; Save current pos

;;;;;;;;;; no 6 ;;;;;;;;;;;

move y:(r6+CH808_COUNT6),b ; Load current pos
move #sqr_table6,y0        ; Load table pos
add y0,b                   ; calc effective adress
move y:(r6+CH808_OUT6),x0   ; Load current out-value
move b,r3                  ; load r3 with adress
move #SQR_FREQ6-1,m3

do #16,ch808_loop6
move x:(r3)+,x1          ; load from table
move x:(r3)+,y1          ; load from table
mpy x0,x1,ax:(r4)+,x1    ; calc out value & load old sum
mpy x0,y1,bx:(r4)+,y1    ; calc out value & load old sum
add x1,a
add y1,b
move a,x:(r5)+           ; Save out
move b,x:(r5)+           ; Save out
ch808_loop6
move r3,b                ; load new adress
sub y0,b                 ; calc pos, table pos already in y1
nop
move b,y:(r6+CH808_COUNT6) ; Save current pos

;;;;;;;;;; done with the sqrs ;;;;;;;;;;;

;;;;;;;;;; filter away ;;;;;;;;;;;
;---Lowpass filter -----

; design the lowpass filter

move y:(r6+CH808_LPFREQ),a
move y:(r6+CH808_LPQ),r2

move #free_y+CH808_VOL1,r4 ; pointer to volume coefficient (a*x) and b2
move #1,m4
```

Appendix B

```
move x:(r2+csqtab),x0
move x:(r2+eqtab),x1

add x0,a1,b
move #free_x+CH808_NOISE_BLOCK,r0; from buffer
move #-1,m0
add x1,b a1,r1
move #free_x+CH808_NOISE_LPQ1,r3
; släpande i from buffer, NOISE_BLOCK-2
move #-1,m3

move b1,r2

move #2,n0
move x:(r1+cstab+2048),x1 ; get values according to freq
move x:(r2+etab+2048),y1 ; get values according to freq

move #free_x+CH808_HISTORY1,r1; pointer to history
move #1,m1

mpy x1,y1,a y1,x0; e*cs
mpy -x0,y1,b ; -e*e
move #free_x+CH808_NOISE_LPQ2,r2
; släpande i from buffer, NOISE_BLOCK-1
move #-1,m2
asl ab,x1 ; 2*e*cs

move y:(r6+CH808_LPOLDY2),y1
move y1,x:(r1)+
move y:(r6+CH808_LPOLDY1),y1
move y1,x:(r1)+

add x1,a ; a,x0
sub #0.50,a ; 1-2*e*cs+e*e = norm
neg a

move b,y:(r4)+ ; init b2
move b,y1

move a,y:(r4) ; a0

; r0: input pointer
; r1: history buffer for y-1,y-2
; r2: pointer to z-1, (r0-1)
; r3: pointer to z-2, (r0-2)
; r4: pointer to a,b2
```

Appendix B

```

move y:(r6+CH808_LPOLDZ2),y0; load old values
move y0,x:(r3)
move y:(r6+CH808_LPOLDZ1),y0
move y0,x:(r2)

```

; filter the signal (24 bit, 24 dB/oct version)

```

movex:(r1)+,x1                                ; get y-2
mpy      x1,y1,a                                ; get y-1
mac      x0,y1,a                                ; y-1 get x   get a0
mac      x1,y0,a                                ; a0*x      get z-1

```

do #32,ch808_lpFilterLoop

```

asl      a                                      x:(r3)+,x1  y:(r4)+,y1 ; shift getb2 and z2
mpy      x1,y1,b                                x:(r2)+,y1 ; get z-1

mac      x0,y1,b                                a,x1      y:(r4)+,y0 ; z-1      get y
mac      x1,y0,b                                a,x:(r1)+ ; a0*y      get y-1
asl      b                                      x:(r1)+,x1 y:(r4)+,y1 ; shift get b2
mpy      x1,y1,a                                x:(r1)+,y1 ; get y-1

mac      x0,y1,a                                x:(r0)-,x1 y:(r4)+,y0 ; y-1 get x   get a0
mac      x1,y0,a                                b,x:(r0)+n0 ; a0*x      get z-1

```

ch808_lpFilterLoop

; r1 points at y-2

```

move x:(r1)+,y1
move y1,y:(r6+CH808_LPOLDY2)
move x:(r1)+,y1
move y1,y:(r6+CH808_LPOLDY1)

```

```

move x:(r2)-,y1
move y1,y:(r6+CH808_LPOLDZ1)
move x:(r2),y1
move y1,y:(r6+CH808_LPOLDZ2)

```

; return to original state

..... LP done

..... now HP

;;hp no 1

;---Hipass filter -----

Appendix B

```
; design the hipass filter

move y:(r6+CH808_HPFREQ),a
move y:(r6+CH808_HPQ),r2

move #free_y+CH808_VOL1,r4
; pointer to volume coefficient (a*x) and b2, five of them
move #4,m4

move x:(r2+csqtab),x0
move x:(r2+eqtab),x1

add x0,a a1,b
move #free_x+CH808_NOISE_LPQ1,r0
;; släpande i from buffer, NOISE_BLOCK-2
move #-1,m0
add x1,b a1,r1

move y:(r6+CH808_HPOLDX2),x1
move x1,x:(r0)+

move b1,r2

move y:(r6+CH808_HPOLDX1),x1
move x1,x:(r0)-

move x:(r1+cstab+2048),x1
move x:(r2+etab+2048),y1

move #free_x+CH808_NOISE_HPQ2,r1
move #-1,m1

mpy x1,y1,a y1,x0; e*cs
mpy -x0,y1,b ; -e*e
move #free_x+CH808_NOISE_HPQ1,r2
move #-1,m2
asl a ; 2*e*cs

move y:(r6+CH808_HPOLDY2),y1
move y1,x:(r2)+
move y:(r6+CH808_HPOLDY1),y1
move y1,x:(r2)

;add x1,a a,x0

;sub #0.50,a ; 1-2*e*cs+e*e = norm
;neg a

move #0.5,x1
move x1,y:(r4)+
```

Appendix B

```
move #-1.0,y0
move y0,y:(r4)+
move x1,y:(r4)+
```

```
move b,y:(r4)+
move a,y:(r4)+
```

```
; filter the signal (24 bit, 12 dB/oct version)
```

```
movex:(r2)-,a ; get y-1
movex:(r0)+,x0 y:(r4)+,y0 ; get x and a0
```

```
do #16,ch808_hpFilterLoop
```

mpy	x0,y0,b	x:(r0)+,x0	y:(r4)+,y0
mac	x0,y0,b	x:(r0)-,x0	y:(r4)+,y0
mac	x0,y0,b	x:(r2)+,x0	y:(r4)+,y0
mac	x0,y0,b	a,x0	y:(r4)+,y0
mac	x0,y0,b	x:(r0)+,x0	y:(r4)+,y0
asl	b	a,x:(r1)+	

mpy	x0,y0,a	x:(r0)+,x0	y:(r4)+,y0
mac	x0,y0,a	x:(r0)-,x0	y:(r4)+,y0
mac	x0,y0,a	x:(r2)+,x0	y:(r4)+,y0
mac	x0,y0,a	b,x0	y:(r4)+,y0
mac	x0,y0,a	x:(r0)+,x0	y:(r4)+,y0
asl	a	b,x:(r1)+	

```
ch808_hpFilterLoop
```

```
; r1 points at y-2
```

```
move x:(r2),y1
move y1,y:(r6+CH808_HPOLDY2)
move a,y:(r6+CH808_HPOLDY1)
move a,x:(r1)+
```

```
move x:(r0),y1
move y1,y:(r6+CH808_HPOLDX1)
move x:-(r0),y1
move y1,y:(r6+CH808_HPOLDX2)
```

```
;;hp no 2
;---Hipass filter -----
```

```
; design the hipass filter
```

```
move y:(r6+CH808_HPFREQ_2),a
move y:(r6+CH808_HPQ_2),r2
```

Appendix B

```
    move #free_y+CH808_VOL1,r4
    ; pointer to volume coefficient (a*x) and b2, five of them
    move #4,m4

    move x:(r2+csqtab),x0
    move x:(r2+eqtab),x1

    add x0,a a1,b
    move #free_x+CH808_NOISE_HPQ1,r0
    ;; släpande i from buffer, NOISE_BLOCK-2
    move #-1,m0                      ;; this is the out buffer from hp 1
    add x1,b a1,r1

    move y:(r6+CH808_HPOLDX2_2),x1
    move x1,x:(r0)+

    move b1,r2

    move y:(r6+CH808_HPOLDX1_2),x1
    move x1,x:(r0)-

    move x:(r1+cstab+2048),x1
    move x:(r2+etab+2048),y1

    move #free_x+CH808_NOISE_LPQ2,r1; we are using the lp buffer to write to now
    move #-1,m1

    mpy x1,y1,a y1,x0; e*cs
    mpy -x0,y1,b    ; -e*e
    move #free_x+CH808_NOISE_LPQ1,r2
    move #-1,m2
    asl a                      ; 2*e*cs

    move y:(r6+CH808_HPOLDY2_2),y1
    move y1,x:(r2)+
    move y:(r6+CH808_HPOLDY1_2),y1
    move y1,x:(r2)

    ;add x1,a                a,x0

    ;sub #0.50,a            ; 1-2*e*cs+e*e = norm
    ;neg a

    move #0.5,x1
    move x1,y:(r4)+
    move #-1.0,y0
    move y0,y:(r4)+
    move x1,y:(r4)+

    move b,y:(r4)+
    move a,y:(r4)+
```

Appendix B

; filter the signal (24 bit, 12 dB/oct version)

```
movex:(r2)-,a
movex:(r0)+,x0 y:(r4)+,y0
```

; get y-1
; get x and a0

do #16,ch808_hpFilterLoop_2

mpy	x0,y0,b	x:(r0)+,x0	y:(r4)+,y0
mac	x0,y0,b	x:(r0)-,x0	y:(r4)+,y0
mac	x0,y0,b	x:(r2)+,x0	y:(r4)+,y0
mac	x0,y0,b	a,x0	y:(r4)+,y0
mac	x0,y0,b	x:(r0)+,x0	y:(r4)+,y0
asl	b	a,x:(r1)+	
mpy	x0,y0,a	x:(r0)+,x0	y:(r4)+,y0
mac	x0,y0,a	x:(r0)-,x0	y:(r4)+,y0
mac	x0,y0,a	x:(r2)+,x0	y:(r4)+,y0
mac	x0,y0,a	b,x0	y:(r4)+,y0
mac	x0,y0,a	x:(r0)+,x0	y:(r4)+,y0
asl	a	b,x:(r1)+	

ch808_hpFilterLoop_2

; r1 points at y-2

```
move x:(r2),y1
move y1,y:(r6+CH808_HPOLDY2_2)
move a,y:(r6+CH808_HPOLDY1_2)
move a,x:(r1)+
```

```
move x:(r0),y1
move y1,y:(r6+CH808_HPOLDX1_2)
move x:- (r0),y1
move y1,y:(r6+CH808_HPOLDX2_2)
```

;;;;;;;;;;done with the filter ;;;;;;;;;;

~~NOISE_BLOCK~~CH808_NOISE_BLOCK,r5; the out data from hp 2 is in
data from hp1 is in NOISE_BLOCK2 ; the

```
move y:(r6+CH808_AMP_ENV),b
```

```
move y:(r6+CH808_HOLD_TIME),a; nr of frames to leave amplitude at max  
left a  
beq ch808_decay ; no, jump to decay ; any  
sub #>1,a  
nop  
move a,y:(r6+CH808_HOLD_TIME)
```

Appendix B

bra ch808_holdout

ch808_decay

move y:(r6+CH808_DECAY_TIME),a; nr of frames left in decay
tst a ; any left?
beq ch808_fadeout ; no, jump to fadeout
sub #>1,a
nop
move a,y:(r6+CH808_DECAY_TIME)

bra ch808_decayout

.....
ch808_holdout

move b,y1 ; note correct amplitude value is already stored in b

do #16,plugin_ch808Loop_hold

move x:(r5)+,y0
move x:(r5)+,x0

; move noise from free_x to y0
; move noise from free_x to x0

mpy y0,y1,a ; this is not needed if we decide to set volume to one at the start
mpy x0,y1,b ; this is not needed if we decide to set volume to one at the start
asr a
asr b

move a,y:(r7)+
move b,y:(r7)+

plugin_ch808Loop_hold

; restore vol to b

move y1,b

rts

.....
ch808_decayout

move y:(r6+CH808_DECAY_AMOUNT),x0

; for first mpy in loop

move b,y1

do #32,plugin_ch808Loop_decay

sub x0,b
move x:(r5)+,y0

; update amplitude envelope
; move noise from free_x to y0
; note correct amplitude value is already stored in b

mpy y0,y1,a
asr a

Appendix B

```
move b,y1
move a,y:(r7)+

plugin_ch808Loop_decay

move b,y:(r6+CH808_AMP_ENV)    ; save envelope

rts

.....
ch808_fadeout

move y:(r6+CH808_FADE_AMOUNT),x0

move b,y1                      ; for first mpy in loop

do #32,plugin_ch808Loop_fade

move x:(r5)+,y0                ; move noise from free_x to y0
                                ; note correct amplitude value is already stored in b

mpy x0,y1,b
mpy y0,y1,a
asr a
move b,y1
move a,y:(r7)+

plugin_ch808Loop_fade

move b,y:(r6+CH808_AMP_ENV)    ; save envelope

rts
```